Tipos de Datos y Bases de Numeración

Fundamentos de Programación / Algoritmos y Programación I
Semana 2 · Clase Teórica · Prof. Servetto

- Bases de numeración y conversiones
- Representación binaria de enteros y complemento a dos
- Números en punto flotante (IEEE 754)
- Caracteres: ASCII, Unicode y UTF-8
- Booleanos
- Limitaciones y buenas prácticas

Bases de Numeración

Todo sistema de numeración posicional usa una base b: el valor de un dígito depende de su posición.

$$\text{Valor} = d_{n-1} \times b^{n-1} + d_{n-2} \times b^{n-2} + \ldots + d_1 \times b^1 + d_0 \times b^0$$

Base	Nombre	Dígitos	Ejemplo	Uso principal
10	Decimal	0-9	42	Uso cotidiano
2	Binario	0, 1	101010	Hardware / memoria
8	Octal	0-7	52	Unix, permisos
16	Hexadecimal	0-9, A-F	2A	Memoria, colores, depuración

$42_{10} = 4\times10^1+2\times10^0 = 40+2$ | $101010_2 = 1\times2^5+0\times2^4+1\times2^3+0\times2^2+1\times2^1+0\times2^0 = 32+8+2 = 42$

Conversión: Decimal → Base b

Algoritmo: División sucesiva

- Dividir el número por la base.
- El resto es el dígito (de derecha a izquierda).
- Continuar con el cociente hasta que sea 0.

Ejemplo: 42_{10} → binario

División	Cociente	Resto (dígito)
42 ÷ 2	21	0 ← LSB
21 ÷ 2	10	1
10 ÷ 2	5	0
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	**1 ← MSB**

Resultado: 101010_2 (leer restos de abajo a arriba)

Ejemplo: 255_{10} → hexadecimal

División	Cociente	Resto
255 ÷ 16	15	15 = F ← LSB
15 ÷ 16	**0**	**15 = F ← MSB**

Resultado: FF_{16}

Dígitos hexadecimales

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	**A**	**B**	**C**	**D**	**E**	**F**

1 dígito hex ↔ 4 bits (nibble). 2 dígitos hex = 1 byte.

Conversión: Base b → Decimal

Algoritmo: Suma ponderada — multiplicar cada dígito por la potencia de la base correspondiente a su posición

Ejemplo: 101011_2 → decimal

Posición	5	4	3	2	1	0
Dígito	1	0	1	0	1	1
2^pos	32	16	8	4	2	1
Valor	**32**	0	**8**	0	**2**	**1**

$32 + 8 + 2 + 1 = 43_{10}$

Ejemplo: $1A3_{16}$ → decimal

Posición	2	1	0
Dígito	1	A (=10)	3
16^pos	256	16	1
Valor	**256**	**160**	**3**

$256 + 160 + 3 = 419_{10}$

Conversión directa binario ↔ hexadecimal: agrupar bits de a 4 (nibbles)

Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Bases 2 y 16: Relación y Equivalencias

$16 = 2^4$ → 1 dígito hex = exactamente 4 bits. Por eso la conversión entre ambas bases es directa y sin división.

Ejemplo: 10110100_2 → hex

Nibble	Bits (grupo)	Valor dec.	Dígito hex
Alto (bits 7-4)	1011	11	B
Bajo (bits 3-0)	0100	4	4

$$10110100_2 = B4_{16}$$

Ejemplo: $3F_{16}$ → binario

Dígito hex	Valor dec.	4 bits
3	3	0011
F	15	1111

$$3F_{16} = 00111111_2$$

¿Por qué usamos hexadecimal?

- Compacto: 1 byte = 2 dígitos hex vs. 8 dígitos binarios
- Directo: sin pasar por decimal (conversión nibble a nibble)
- Usos reales: direcciones de memoria (0xFFFF), colores web (#1C2B3A), opcodes de CPU, depuración

Tipos de Datos Básicos: Panorama General

A nivel de máquina todo es una secuencia de bits. El tipo de dato es la interpretación que se le da.

Números Enteros

- Con signo / sin signo
- 8, 16, 32, 64 bits
- Compl. a dos (con signo)

Punto Flotante

- Precisión simple (32b)
- Precisión doble (64b)
- Estándar IEEE 754

Caracteres

- ASCII (7/8 bits)
- Unicode / UTF-8
- UTF-16, UTF-32

Booleanos

- Verdadero / Falso
- 1 bit lógico
- 8+ bits físicos

Conceptos fundamentales

Concepto	Definición
Bit	Unidad mínima de información (0 o 1)
Byte	8 bits — unidad básica de almacenamiento
Word (palabra)	Unidad natural del procesador (32 o 64 bits en sistemas modernos)
Tipo de dato	Interpretación específica que se da a una secuencia de bits

Números Enteros: Representación Binaria

Sin signo (Unsigned)

Bits	Mínimo	Máximo	Fórmula
8	0	255	$2^8 - 1$
16	0	65,535	$2^{16} - 1$
32	0	4,294,967,295	$2^{32} - 1$
64	0	18.4×10^{18}	$2^{64} - 1$

Con signo: Complemento a dos

Bits	Mínimo	Máximo
8	−128	+127
16	−32,768	+32,767
32	−2,147,483,648	+2,147,483,647
64	-9.2×10^{18}	$+9.2 \times 10^{18}$

¿Cómo se obtiene el complemento a dos de un número negativo?

Paso	Operación	Ejemplo: −42 en 8 bits
1. Representar |n|	42 en binario puro (8 bits)	`00101010`
2. Invertir bits	NOT bit a bit (complemento a uno)	`11010101`
3. Sumar 1	Sumar 1 al resultado anterior	**`11010110 → −42`**

El bit más significativo (MSB) indica el signo: 0 = positivo · 1 = negativo

Limitaciones Intrínsecas: Overflow y Underflow

Los tipos enteros tienen rango finito. Superar ese rango produce comportamientos incorrectos — errores silenciosos frecuentes.

Overflow — exceder el máximo

Situación	Resultado
int8 sin signo: 255 + 1	→ `0 (vuelve al mínimo)`
int8 con signo: 127 + 1	→ `-128 (wrapping)`
Python int	→ **Precisión arbitraria (no hay overflow)**

Underflow — exceder el mínimo

- uint8: 0 – 1 → 255 (resultado incorrecto)
- En flotante: un número muy pequeño se redondea a cero

¿Qué pasa en Python?

Tipo Python	Comportamiento	Ejemplo
int	Precisión arbitraria — sin overflow	`2**100 funciona`
float	64 bits IEEE 754 — sí puede ser inf o NaN	`1e400 → inf`
array (numpy)	Tipos fijos — overflow posible	`np.uint8(255)+1 → 0`

El overflow es fuente frecuente de bugs y vulnerabilidades. Comprender el tipo subyacente es esencial.

Punto Flotante: Estándar IEEE 754

Estructura del número en punto flotante: ± mantisa × 2^exponente

Formato	Total	Signo	Exponente	Mantisa	Rango aprox.	Precisión decimal
Simple	32 bits	1 bit	8 bits	23 bits	$\pm 3.4 \times 10^{38}$	~7 dígitos
Doble	64 bits	1 bit	11 bits	52 bits	$\pm 1.7 \times 10^{308}$	~15-16 dígitos

Ejemplo: 9.3 en decimal → nunca se representa exactamente en binario

Paso	Operación	Resultado
1. Binario de 9	$9 = 8+1 = 2^3+2^0$	1001_2
2. Fracción 0.3	0.3 × 2 → periódica: 0100110011...	**Infinita en binario**
3. Normalizar	$1001.0100... = 1.0010100... \times 2^3$	`Exp real = +3`
4. Almacenado	Sesgo 127 → exp = 130 = 10000010_2	**Mantisa truncada: error**
5. Reconversión	9.30000019073486328125	**≠ 9.3 (error ~2×10^{-7})**

Consecuencia: (0.1 + 0.2) == 0.3 → False en Python (y en casi todos los lenguajes)

Caracteres: ASCII, Unicode y UTF-8

ASCII

- 7 bits → 128 caracteres posibles
- Cubre: inglés, dígitos, signos de puntuación, control
- Limitación: no sirve para otros idiomas

Rango	Contenido	Ejemplo
0–31	Caracteres de control	\n, \t, \0
48–57	Dígitos '0'–'9'	`'0'=48, '9'=57`
65–90	Mayúsculas 'A'–'Z'	`'A'=65, 'Z'=90`
97–122	Minúsculas 'a'–'z'	`'a'=97 = 'A'+32`

Unicode y UTF-8

Codificación	Bytes/carácter	Característica
UTF-8	1 a 4 (variable)	Compatible con ASCII · web · Unix
UTF-16	2 o 4	Windows, Java, JS · problema endianness
UTF-32	Siempre 4	Índice directo · desperdicia espacio

- Unicode cubre + de 149.000 caracteres (code points 0 a 0x10FFFF)
- Incluye todos los alfabetos, símbolos matemáticos y emojis
- **Python 3: str es Unicode. Usar encoding='utf-8' siempre al abrir archivos.**

Caracteres: ASCII, Unicode y UTF-8

¿Por qué importa en programación?	
Problema	**Descripción**
Mojibake	Leer un archivo UTF-8 como Latin-1 produce caracteres ilegibles (ñ → Ã±)
Indexación	En UTF-8 len('á') puede ser > 1 byte. No confundir bytes con caracteres.
Comparaciones	ord('A')=65, chr(65)='A'. Las operaciones con caracteres son operaciones numéricas.

Booleanos · Limitaciones Extrínsecas

Tipo Booleano

Aspecto	Detalle
Valores lógicos	Verdadero (True) / Falso (False)
Almacenamiento	1 bit lógico, pero mínimo 8 bits físicos (1 byte)
Representación	0 = False · cualquier valor ≠ 0 = True (en C/C++)
Python bool	`Subclase de int. True==1, False==0. True+True == 2`

Limitaciones Extrínsecas

Limitación	Descripción
Endianness	Little-endian (Intel x86): byte menos significativo primero. Big-endian (red): al revés. Mismo valor, diferente orden en memoria.
Tamaño variable	'int' en C puede ser 16, 32 o 64 bits según la plataforma.
Alineación	El hardware exige datos en direcciones múltiplo de su tamaño (padding).
Codificación	Archivos binarios entre sistemas pueden ser ilegibles sin especificar endianness y codificación.

Booleanos · Limitaciones Extrínsecas

Tipos primitivos en Python vs. tipos de máquina

Python	Tipo subyacente	Observación
`int`	Precisión arbitraria	No tiene overflow pero usa más memoria al crecer
`float`	double IEEE 754 (64 bits)	`0.1 + 0.2 ≠ 0.3`
`str`	Unicode (UTF-8 internamente)	Inmutable. Cada carácter es un code point Unicode.
`bool`	Subclase de int	`True == 1, False == 0`

Tabla Comparativa de Tipos de Datos

Tipo	Tamaño típico	Rango / Capacidad	Limitación principal	Portabilidad
int 8 bits (sin signo)	1 byte	0 a 255	Overflow muy fácil	Portable
int 8 bits (con signo)	1 byte	−128 a +127	Overflow fácil	Portable
int 16 bits	2 bytes	0–65,535 / −32,768–32,767	Overflow en cálculos	Endianness
int 32 bits	4 bytes	0–4.3G / ±2.1G	Overflow en app. grandes	Endianness
int 64 bits	8 bytes	$\pm 9.2 \times 10^{18}$	Overflow raro	Endianness, alineación
float simple	4 bytes	$\pm 3.4 \times 10^{38}$ (~7 díg.)	Precisión limitada, NaN, ∞	IEEE 754 portable
float doble	8 bytes	$\pm 1.7 \times 10^{308}$ (~15 díg.)	Errores acumulados, comparaciones	IEEE 754 portable
char ASCII	1 byte	128 caracteres	Solo inglés básico	Muy portable
char UTF-8	1-4 bytes	Todo Unicode	Long. variable, indexación	Portable, sin endianness
bool	1 bit / ≥1 byte	True / False	7 bits desperdiciados	Varía (0/1 vs 0/≠0)

Buenas Prácticas y Conexión con el Lenguaje de Máquina

Buenas prácticas con tipos de datos

Regla	Descripción
Elegir el tipo correcto	Usar el tipo adecuado al problema: int para contadores, float para magnitudes continuas.
No comparar floats directamente	`Usar tolerancia: abs(a - b) < 1e-9 en vez de a == b.`
Validar rangos	Verificar que los valores no excedan los límites del tipo antes de operar.
Siempre especificar encoding	`open('f', encoding='utf-8'). Nunca asumir la codificación del sistema.`
Documentar unidades	Indicar en el código la unidad de cada variable: metros, segundos, grados.

[Holístico] Lenguaje de Máquina

Concepto	Relación con tipos de datos
Opcode (código de operación)	Identifica la instrucción (ADD, MOV, CMP…)
Operandos	Especifican dónde están los datos (registro, memoria, inmediato)
Modos de direccionamiento	Inmediato, directo, indirecto, indexado…
Relación dato–representación	La CPU opera sobre bits: el tipo decide cómo interpretarlos (ADD enteros ≠ ADD flotantes)
ALU	Unidades separadas para enteros y para punto flotante (FPU)

Literate Programming: documentar el tipo, el rango válido y las unidades antes de codificar

```
# velocidad: float [m/s], rango válido 0.0 – 340.0 (mach 1 en aire estándar)
# temperatura: float [°C], rango válido -273.15 – 5778
```

Síntesis de la Clase

Tema	Idea central
Bases de numeración	Toda base posicional usa suma ponderada. Dec→base: divisiones sucesivas. Base→dec: suma ponderada.
Binario ↔ Hexadecimal	1 dígito hex = 4 bits. La conversión es nibble a nibble, sin pasar por decimal.
Enteros	Sin signo: binario puro. Con signo: complemento a dos. Rango finito → overflow posible.
Punto flotante	IEEE 754: signo + exponente (con sesgo) + mantisa. 0.1 en binario es periódico → error ineludible.
Caracteres	ASCII: 128 car. con 7 bits. Unicode: todo. UTF-8: variable, compatible con ASCII, recomendado.
Booleanos	1 bit lógico, ≥ 8 físicos. En Python: subclase de int.
Limitaciones	Intrínsecas: rango, precisión. Extrínsecas: endianness, tamaño variable, codificación.

Actividades de extracursado: lectura 'Tipos de Datos Básicos' · 'Lenguajes de Máquina y Evolución de la Programación' · ejercicios Moodle · autoevaluación